

СОФТУЕРНИ АРХИТЕКТУРИ

ВАЛИДЕН XML



1. ДОБРЕ КОНСТРУИРАН XML

- Добре конструираните XML данни със сигурност използват правилен XML синтаксис и имат подходящо свързана йерархична структура.
- Това е достатъчна информация за статичните вътрешни приложения, когато XML данните са компютърно генерирани.

ДОБРЕ КОНСТРУИРАН XML

- Приложенията са отговорни за всички проверки на структурата и съдържанието, обработката на грешките и интерпретирането на данните – твърдо кодирани в приложенията за получаване и изпращане на данни, чрез обща спецификация.

ДОБРЕ КОНСТРУИРАН XML

- Всяка промяна на XML структурата се извършва на три места:
 - в спецификацията
 - в приложението за приемане
 - в приложението за изпращане
- Когато трансфера на данните е в рамките на домейна не е необходимо валидизиране на данните – достатъчен е добре конструираният XML.

ДОБРЕ КОНСТРУИРАН XML

- Изисквания към XML приложенията:
 - да описват структурата на документа
 - да предават структурата на документа на други приложения и хора
 - да проверяват за наличие на задължителни елементи
 - да проверяват за включване на непозволени елементи
 - да задават стойности за дървовидна структура, съдържание на елементи и атрибути
 - да осигуряват стойности по подразбиране за неопределените атрибути
 - да използват стандартни формати за документа и структурата от данни

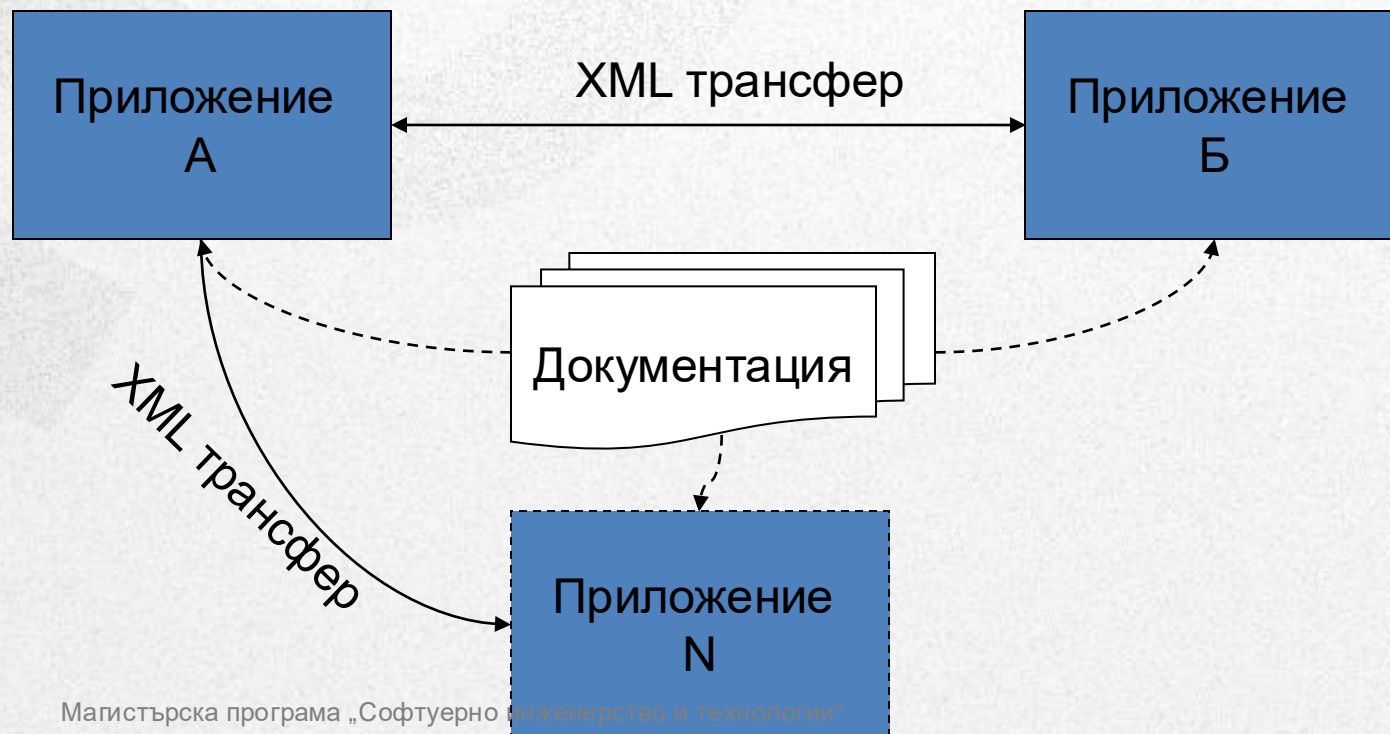
ДОБРЕ КОНСТРУИРАН XML

- Всички тези функции могат да се осигурят от специфичен код в двойка сътрудничащи си приложения и съпътстващата ги документация.



ДОБРЕ КОНСТРУИРАН XML

- Когато XML данните са свободно достъпни (между много приложения) поддръжката на тези функции във всяко приложение се превръща в експоненциален проблем.



2. ВАЛИДЕН XML

- Отделянето на описанието на данните от индивидуалните приложения позволява всички коопериращи се приложения да споделят едно-единствено описание на данните известно под името **XML речник**.
- Група от XML документи, които споделят общ XML речник се наричат **документен тип**.
- Всеки отделен документ, отговарящ на този тип, е **документна инстанция**.

ВАЛИДЕН XML

- Спецификацията на XML 1.0 съдържа стандартизиран метод за описание на XML документни типове: **дефиниция на документен тип – DTD** (Document Type Definition)
- DTD дефинициите са XML документи, които могат да се вграждат в XML документ, съдържащ данни, или да съществуват като отделен документ.
- DTD дефинициите описват правилата за структуриране на документа, включените елементи, възможните типове данни и стойности по подразбиране.

ВАЛИДЕН XML

- Валидизиращи парсери – извършват по-конкретна верификация за определяне на съдържанието на XML документите като **валидно**.
- Тази валидизация е свързана със сравняване на съдържанието на документа с асоцииран шаблон под формата на DTD.

ВАЛИДЕН XML

- *Добре конструиран XML документ е този, който отговаря на основния синтаксис и структурни правила от спецификацията на XML 1.0.*
- *Валиден XML документ е добре конструиран документ, който освен това отговаря на синтактични, структурни и други правила, дефинирани в една DTD дефиниция.*

ВАЛИДЕН XML

- Наличието на централно дефинирани XML данни и стандартни методи за валидизиране води до следните предимства:
 - изнасяне на описанието (DTD) и валидиращия код (валидиращ XML парсер) извън множеството специфични приложения
 - опростяване на приложения код
 - подобряване на производителността (валидиращия парсер е оптимизиран)
 - подобряване на сигурността

3. DTD ДЕФИНИЦИИ

- DTD дефинициите произлизат от много по-сложните версии, създадени и използвани в SGML.
- DTD използват формална граматика за описание на структурата и синтаксиса на XML документа, включително и възможните стойности за голяма част от съдържанието на документа.

DTD ДЕФИНИЦИИ

- DTD дефинициите:
 - осигуряват формална и пълна дефиниция на XML речника
 - са споделяемо описание на структурата на XML документа
 - са начин за валидизиране на XML документи и ограничаване на тяхното съдържание
 - са ограничени до една дефиниция за документен екземпляр

DTD ДЕФИНИЦИИ

- Само една DTD дефиниция може да се свърже с даден документ. Тя може да се раздели на две части:
 - вътрешно подмножество (включено в документа)
 - външно подмножество (набор от декларации, намиращи се в отделен документ)
- Няма изискване за използване и на двете подмножества.
- DTD декларациите от вътрешното подмножество имат приоритет пред тези от външното.

DTD ДЕФИНИЦИИ

- Външни DTD дефиниции се използват, когато се споделят между множество документи.
- Вътрешни DTD дефиниции се използват за внасяне на малки промени в стандартни DTD дефиниции за собствени цели

DTD ДЕФИНИЦИИ

- Асоцииране на DTD дефиниция с XML документ

```
<!DOCTYPE TestDocType SYSTEM "TestDTD.dtd">
```

- Основно DTD форматиране:

```
<!ключова_дума параметър1 параметър2 ... >
```

DTD ДЕФИНИЦИИ

- КЛЮЧОВИ думи:
 - **ELEMENT** – декларира име на XML елементен тип и неговите възможни поделемени
 - **ATTLIST** – декларира имена на XML елементи плюс разширени и/или подразбиращи се стойности за атрибутите
 - **ENTITY** декларира специални знакови препратки, текстови макроси, както и друго повтарящо се съдържание от външни източници
 - **NOTATION** – декларира външно не-XML съдържание (напр. двоични данни за изображение) и външни приложения, които обработват това съдържание

DTD ДЕФИНИЦИИ

- Декларация за елементен тип

<!ELEMENT *име категория*>

<!ELEMENT *име (модел_на_съдържание)*>

КАТЕГОРИИ

- **ANY** – елементният тип може да съдържа всякакви добре конструирани XML данни

```
<!ELEMENT element-name ANY>
```

```
<!ELEMENT note ANY>
```

```
<note>Текст на бележката</note>
```

```
<note>  
    <content>  
        Съдържание на бележката  
    </content>  
</note>
```

КАТЕГОРИИ

- **EMPTY** – елементният тип **не** може да съдържа нито текст, нито дъщерни елементи – разрешени са само елементни атрибути

```
<!ELEMENT element-name EMPTY>
```

```
<!ELEMENT br EMPTY>
```

```
<br/>
```

```
<br lines="5"/>
```

МОДЕЛИ НА СЪДЪРЖАНИЕ

- Модели на съдържание – използват се за описание на структурата и съдържанието на даден елементен тип. Може да бъде:

- знакови данни (PCDATA съдържание)

```
<!ELEMENT description (#PCDATA)>
```

- един или повече дъщерни елементни типове

```
<!ELEMENT html (head, body)>
```

- комбинация от горните две – смесено съдържание

```
<!ELEMENT p (#PCDATA | p | ul | dl | table | h1|h2|h3)*>
```

МОДЕЛИ НА СЪДЪРЖАНИЕ

- Знакови данни (#PCDATA). PCDATA означава текст, който е преминал процеса на парсиране (Parsed Character Data). Т.е. не се допускат специални символи в него, като: <, &

```
<!ELEMENT element-name (#PCDATA)>
```

```
<!ELEMENT note (#PCDATA)>
```

```
<note>Текст на бележката</note>
```

МОДЕЛИ НА СЪДЪРЖАНИЕ

- Един или повече дъщерни елементни типове

```
<!ELEMENT element-name (child)>
```

```
<!ELEMENT element-name (child1, child2, ...)>
```

```
<!ELEMENT message (to, from, heading, body)>
```

```
<message>
```

```
  <to>...</to>
```

```
  <from>...</from>
```

```
  <heading>...</heading>
```

```
  <body>...</body>
```

```
</message>
```

МОДЕЛИ НА СЪДЪРЖАНИЕ

- Честота на срещане – точно един път

```
<!ELEMENT element-name (child-name)>
```

```
<!ELEMENT note (message)>
```

```
<note>  
    <message>...</message>  
</note>
```

```
<note>  
    <message>...</message>  
    <message>...</message>  
</note>
```



МОДЕЛИ НА СЪДЪРЖАНИЕ

- Честота на срещане – един или повече пъти (+)

```
<!ELEMENT element-name (child-name+)>
```

```
<!ELEMENT note (message+)>
```

```
<note>  
  <message>...</message>  
  <message>...</message>  
</note>
```

```
<note>  
  
</note>
```



МОДЕЛИ НА СЪДЪРЖАНИЕ

- Честота на срещане – нула или един път (?)

```
<!ELEMENT element-name (child-name?)>
```

```
<!ELEMENT note (message?)>
```

```
<note> </note>
```

```
<note>  
    <message>...</message>  
</note>
```

```
<note>  
    <message>...</message>  
    <message>...</message>  
</note>
```



МОДЕЛИ НА СЪДЪРЖАНИЕ

- Честота на срещане – нула или повече пъти (*)

```
<!ELEMENT element-name (child-name*)>
```

```
<!ELEMENT note (message*)>
```

```
<note>
```

```
</note>
```

```
<note>
```

```
    <message>...</message>
```

```
    <message>...</message>
```

```
</note>
```

МОДЕЛИ НА СЪДЪРЖАНИЕ

- Алтернативно съдържание (|)

```
<!ELEMENT element-name (child1 | child2)>
```

```
<!ELEMENT note (to, from, header, (message|body))>
```

```
<note>  
  <to> ... </to>  
  <from> ... </from>  
  <header> ... </header>  
  <message> ... </message>  
</note>
```

```
<note>  
  <to> ... </to>  
  <from> ... </from>  
  <header> ... </header>  
  <body> ... </body>  
</note>
```

```
<note>  
  ...  
  <message>...</message>  
  <body>...</body>  
</note>
```

HE

МОДЕЛИ НА СЪДЪРЖАНИЕ

- Смесено съдържание

```
<!ELEMENT element-name (#PCDATA | child1 | child2 | ...) *>
```

```
<!ELEMENT note (#PCDATA|to|from|header|message) *>
```

<note>

<to> ... </to>

<from> ... </from>

Свободен текст

<header> ... </header>

<to> ... </to>

<message> ... </message>

<message> ... </message>

</note>

АТРИБУТИ

```
<!ATTLIST element-name attribute-name attribute-type attribute-value>
```

```
<!ATTLIST payment type CDATA "check">
```

ТИП НА АТРИБУТ

Тип	Описание
CDATA	Символни данни
(<i>en1</i> <i>en2</i> ..)	Стойността може да е само от изброените елементи
ID	Уникален идентификатор
IDREF	Стойността е ID на друг елемент
IDREFS	Стойността е списък от ID на други елементи
...	...

СТОЙНОСТ НА АТРИБУТ

Стойност	Обяснение
<i>value</i>	Стойност по подразбиране
#REQUIRED	Атрибута е задължителен
#IMPLIED	Атрибута не е задължителен
#FIXED <i>value</i>	Стойността на атрибута е фиксирана

DTD ДЕФИНИЦИИ

- Ограничения:
 - DTD дефинициите не са разширяеми
 - Само една DTD дефиниция може да бъде асоциирана с даден документ
 - DTD дефинициите не работят добре с пространствата от имена
 - Много слабо типизиране на данните
 - Ограничени описания за моделите на съдържание
 - Липса на обектно-ориентирано наследяване на обекти
 - Един документ може да предефинира/игнорира външна DTD дефиниция
 - Не-XML синтаксис
 - Няма DOM поддръжка
 - Сравнително малко, стари и скъпи инструменти

ВАЛИДЕН XML: СХЕМИ

- Разликите между използване на DTD дефиниции и XML схеми:
 1. Синтаксис:
 - DTD – EBNF + псевдо XML
 - XML Schema – XML 1.0
 2. Инструменти – използват почти всеки XML инструмент
 3. Поддръжка на DOM

ВАЛИДЕН XML: СХЕМИ

- 4. Силни модели на съдържанието
- 5. Много добро типизиране на данните (низове, числа, дата/време и структури)
- 6. Област на имената – глобални и локални
- 7. Разширяемост – неограничена
- 8. Динамични схеми

XML CXEMII

- Корен на схемата: schema

```
<xs:schema  
  xmlns:xs="http://www.w3.org/2001/XMLSchema"  
  elementFormDefault="qualified"  
  attributeFormDefault="unqualified">
```

XML СХЕМИ

- Дефиниране на единичен елемент

```
<xs:element name="xxx" type="yyy"/>
```

- Типове:

- xs:string
- xs:decimal
- xs:integer
- xs:boolean
- xs:date
- xs:time

```
<xs:element name="lastname" type="xs:string"/>  
<xs:element name="age" type="xs:integer"/>  
<xs:element name="dateborn" type="xs:date"/>
```

```
<lastname>Petrov</lastname>  
<age>20</age>
```

```
<age>двадесет</age>
```

НЕ

XML CXEMИ

- Фиксирани и стойности по подразбиране

```
<xs:element name="xxx" type="yyy" default="defValue"/>
```

```
<xs:element name="xxx" type="yyy" fixed="fixedValue"/>
```

```
<xs:element name="color" type="xs:string" default="red"/>
```

```
<xs:element name="pi" type="xs:decimal" fixed="3.14"/>
```

```
<color/>  
<pi/>
```

```
<color>green</color>  
<pi>3.14</pi>
```

```
<pi>3.15</pi>
```

HE

ОГРАНИЧЕНИЯ

- Ограниченията се използват за да се дефинират приемливи стойности за XML елементите и атрибутите

```
<xs:element name="age">
  <xs:simpleType>
    <xs:restriction base="xs:integer">
      <xs:minInclusive value="0"/>
      <xs:maxInclusive value="120"/>
    </xs:restriction>
  </xs:simpleType>
</xs:element>
```

ОГРАНИЧЕНИЯ

Ограничение	Описание
enumeration	Списък от допустими стойности
fractionDigits	Specifies the maximum number of decimal places allowed. Must be equal to or greater than zero
length	Specifies the exact number of characters or list items allowed. Must be equal to or greater than zero
maxExclusive	Specifies the upper bounds for numeric values (the value must be less than this value)
maxInclusive	Specifies the upper bounds for numeric values (the value must be less than or equal to this value)
maxLength	Specifies the maximum number of characters or list items allowed. Must be equal to or greater than zero
minExclusive	Specifies the lower bounds for numeric values (the value must be greater than this value)
minInclusive	Specifies the lower bounds for numeric values (the value must be greater than or equal to this value)
minLength	Specifies the minimum number of characters or list items allowed. Must be equal to or greater than zero
pattern	Defines the exact sequence of characters that are acceptable
totalDigits	Specifies the exact number of digits allowed. Must be greater than zero
whiteSpace	Specifies how white space (line feeds, tabs, spaces, and carriage returns) is handled